🎯 Project Overview
You're building a distributed Reddit clone using Gleam (a functional programming language that runs on the Erlang VM). The project has two main parts:
Engine (Backend) - The Reddit server
Client/Simulator - Simulates thousands of users interacting with the system

---

📐 Architecture Deep Dive
The Actor Model (Core Concept)
This project uses the Actor Model for concurrency - a fundamental pattern in Erlang/Gleam:
Each "actor" is an independent process with its own state
Actors communicate by sending messages (no shared memory)
Actors process messages one at a time (no race conditions!)
If an actor crashes, it doesn't affect others
Think of it like a post office: each actor is a mailbox that receives letters (messages) and responds to them one at a time.

📦 Engine Components (Backend)
1. Core Engine (core.gleam)
What it does: Central coordinator that routes requests to the right subsystem.
gleam
pub type EngineState {
  EngineState(
    user_registry: Subject(user_registry.UserRegistryMessage),
    subreddit_manager: Subject(subreddit_manager.SubredditMessage),
    post_storage: Subject(post_storage.PostStorageMessage),
    vote_tracker: Subject(vote_tracker.VoteTrackerMessage),
    active_connections: Dict(Id, Subject(EngineResponse)),
)
}
```

**How it works:**
- **Starts 4 subsystem actors** when initialized
- **Routes incoming requests** to appropriate subsystems
- **Tracks active connections** (which users are online)
- Acts like a **dispatcher/orchestrator**

**Example flow:**
```
Client sends "CreatePost" → Core → Routes to post_storage
                                 → Also notifies subreddit_manager
```

---

2. User Registry (user_registry.gleam)
What it does: Manages all user-related operations.
State:
gleam
UserRegistryState(
```

```
  users: Dict(Id, User),            // user_id → User data
  usernames: Dict(String, Id),      // username → user_id lookup
  direct_messages: Dict(Id, List(DirectMessage)), // user_id → their messages
)
```

Operations:

RegisterUser: Creates new user, hashes password, generates unique ID

LoginUser: Verifies credentials

SendDirectMessage: Stores DMs in recipient's inbox

UpdateUserKarma: Tracks user reputation points

Subscribe/Unsubscribe: Manages which subreddits user follows

Key insight: All user data is in-memory (fast but volatile). In production, you'd add persistence (database).

---

3. Subreddit Manager (subreddit_manager.gleam)

What it does: Manages subreddit communities.

State:

```
gleam
SubredditManagerState(
  subreddits: Dict(Id, Subreddit),      // All subreddits
  subreddit_names: Dict(String, Id),    // Name → ID lookup
  members: Dict(Id, List(Id)),          // subreddit_id → list of member IDs
)
```

Operations:

CreateSubreddit: Validates name (3-21 chars), creates community

JoinSubreddit: Adds user to member list, increments count

LeaveSubreddit: Removes user, decrements count

IncrementPostCount: Called when new post is created

Zipf Distribution Connection: Your requirement mentions simulating Zipf distribution - this means some subreddits should have way more members than others (like real Reddit where r/funny has millions but r/obscure_hobby has dozens).

---

4. Post Storage (post_storage.gleam)

What it does: Manages posts and comments (the content!).

State:

```
gleam
PostStorageState(
  posts: Dict(Id, Post),                // All posts
  comments: Dict(Id, Comment),          // All comments
  post_comments: Dict(Id, List(Id)),    // post_id → comment IDs
  subreddit_posts: Dict(Id, List(Id)),  // subreddit_id → post IDs
)
```

Operations:

- **CreatePost**: Validates title/content, generates ID, stores post
- **CreateComment**: Supports hierarchical comments (replies to replies!)
- **GetFeed**: Returns posts using feed algorithms (Hot, New, Top, Rising)
- **UpdatePostVotes**: Called by vote_tracker when someone votes

Comment Hierarchy Example:
```
Post: "Check out Gleam!"
 ├─ Comment 1: "Nice!" (depth=0)
|  └─ Comment 2: "I agree!" (depth=1)
|     └─ Comment 3: "Me too!" (depth=2)
└─ Comment 4: "Cool language" (depth=0)
```
Each comment tracks its parent_comment_id and depth.

---

5. Vote Tracker (vote_tracker.gleam)
What it does: Manages upvotes/downvotes and karma calculation.
State:
```
gleam
VoteTrackerState(
  votes: Dict(String, Vote),            // "user_id:target_id" → Vote
  target_votes: Dict(Id, #(Int, Int)),    // target_id → (upvotes, downvotes)
)
```

Key logic:
- **Prevents duplicate votes** (same user can't upvote twice)
- **Allows vote changing** (upvote → downvote updates both counts)
- **Calculates karma**: `score = upvotes - downvotes`

Example:
```
User1 upvotes Post1:
  votes["user1:post1"] = Vote{type: Upvote}
  target_votes["post1"] = (1, 0)  // 1 upvote, 0 downvotes

User1 changes to downvote:
  votes["user1:post1"] = Vote{type: Downvote}
  target_votes["post1"] = (0, 1)  // 0 upvotes, 1 downvote
```

---

6. Feed Generator (feed_generator.gleam)
What it does: Implements Reddit's famous ranking algorithms!
Algorithms:
Hot (Default front page):
```
gleam
   score = log10(abs(upvotes - downvotes)) + sign * (age_seconds / 45000)
```
Balances popularity with recency
Old posts decay over time
Highly upvoted posts stay visible longer
New: Simple chronological (newest first)
Top: Pure score (upvotes - downvotes), ignores age
Rising: Recent posts with growing engagement
```
gleam
```

score = engagement / (age_hours + 1)
Only considers posts < 24 hours old
Good for discovering trending content
User Feed vs All Posts:
User Feed: Only shows posts from subscribed subreddits
All Feed: Shows everything (for discovery)

🤖 Simulator/Client Components
7. Simulator (simulator.gleam)
What it does: Simulates hundreds/thousands of concurrent users.
Configuration:

```
gleam
SimulationConfig(
  num_users: 50,            // How many simulated users
  num_subreddits: 10,       // How many communities
  zipf_skewness: 1.0,       // Distribution skew (higher = more uneven)
  actions_per_user: 10,     // Actions each user performs
)
```

4 Simulation Phases:
Phase 1: Register Users

```
gleam
register_users(state)
```

→ Creates 50 users: user1, user2, ..., user50
→ Each gets a client actor to communicate with engine
→ Sends RegisterUser requests to engine
Phase 2: Create Subreddits

```
gleam
create_subreddits(state)
```

→ Creates subreddits: r/gleam, r/programming, etc.
→ Random users become moderators/creators
→ Sends CreateSubreddit requests
Phase 3: Users Join Subreddits (Zipf Distribution)

```
gleam
users_join_subreddits(state)
```

→ Each user joins 1-5 random subreddits
→ Should implement Zipf: most users join popular subs
→ Currently random (TODO: add Zipf selection)
Phase 4: Simulate Activities

```
gleam
simulate_user_activities(state)
```

→ Each user performs random actions:
- 40% create posts
- 25% create comments
- 20% cast votes
- 10% view feed
- 5% send DMs

Why this matters for your project: The requirement says "simulate as many users as you can" - this simulator lets you test with 1000s of concurrent actors, each behaving like a real user!

8. User Actor (user_actor.gleam)
What it does: Represents a single user's behavior (not heavily used in current simulator).
State:
gleam
UserActorState(
  username: String,
  engine: Subject(EngineMessage),
  subscribed_subreddits: List(Id),
  posts_created: Int,
  comments_created: Int,
  votes_cast: Int,
  is_connected: Bool,
  action_count: Int,
)
```

**Actions:**
- Register, Login, CreatePost, CreateComment, VotePost
- JoinSubreddit, LeaveSubreddit, ViewFeed, SendMessage

This is like a more structured version of the simulated users in `simulator.gleam`.

---

## 🔄 Message Flow Example

Let's trace what happens when a user creates a post:
```
1. Simulator creates user "alice"
   → simulator.gleam creates SimulatedUser

2. Alice creates a post:
   → core.handle_request(engine, client, CreatePost(...))

3. Core routes to post_storage:
   → handle_client_request matches CreatePost
   → Calls post_storage.create_post(...)

4. Post Storage validates & stores:
   → Validates title (1-300 chars)
   → Validates content (<40k chars)
   → Generates unique post_id
   → Stores in posts Dict
   → Adds to subreddit_posts mapping
   → Sends PostCreated response

5. Core notifies subreddit_manager:
```
```

→ Increments post count for subreddit

6. Response flows back:
→ post_storage → core → client actor → alice
All of this happens asynchronously! Multiple users can post simultaneously without blocking each other.

🎯 How This Relates to Your Requirements
✅ Implemented Requirements:
✓ Register account - user_registry.gleam handles registration
✓ Create & join subreddits - subreddit_manager.gleam
✓ Post in subreddit - post_storage.gleam with validation
✓ Hierarchical comments - Comments track parent_comment_id and depth
✓ Upvote/downvote + karma - vote_tracker.gleam calculates scores
✓ Get feed - feed_generator.gleam with Hot/New/Top/Rising
✓ Direct messages - user_registry.gleam stores DMs
✓ Simulate many users - simulator.gleam can spawn thousands
✓ Simulate connections/disconnections - Tracked in active_connections
⚠️ Partially Implemented:
⚠️ Zipf distribution - Infrastructure exists (zipf.gleam) but not fully integrated
Currently users join subreddits randomly
Should use zipf.gleam to make popular subs more popular
⚠️ Repost detection - Posts have is_repost flag but no enforcement
📊 Performance Measurement (Still TODO):
Your requirement says "measure various aspects":
Message latency (time from request → response)
Throughput (posts/comments per second)
Memory usage per user
Feed generation time
Need to add:
Timing instrumentation
Metrics collection actor
Performance reporting

🧩 Key Gleam/Erlang Concepts Used
1. Pattern Matching
gleam
case message {
  CreatePost(...) -> // handle post creation
  CreateComment(...) -> // handle comment
  VotePost(...) -> // handle vote
}
No if/else chains - just match the message shape!
2. Immutable Data
gleam
let new_state = PostStorageState(..state, posts: new_posts)
Never mutate - always create new versions. The ..state copies existing fields.

3. Result Type (No Exceptions!)
gleam
Result(Post, String) // Either success with Post, or error with String
Forces explicit error handling.
4. Actor Message Passing
gleam
process.send(actor, Message) // Non-blocking send
actor.continue(new_state)   // Process next message
5. Supervision (Implicit)
If an actor crashes, the system can restart it without affecting others.

🚀 What's Missing for Full Implementation
Persistence Layer
Currently all in-memory (data lost on restart)
Need: ETS tables or PostgreSQL integration
Metrics Collection
Add timing to every operation
Track throughput, latency, memory
Full Zipf Integration
Use zipf.gleam in users_join_subreddits
Make popular subreddits get exponentially more members
Connection Cycling
Simulate users connecting/disconnecting over time
Test resilience
WebSocket/HTTP Layer (Part II)
Currently just direct actor communication
Part II will add REST API
Main Entry Point
Need main.gleam to start engine + simulator
Parse command-line args for config

🎓 Why This Architecture is Brilliant
Fault Tolerance: One actor crashing doesn't kill the system
Scalability: Can run on multiple machines (Erlang's superpower)
No Locks: Message passing eliminates race conditions
Hot Code Reload: Can update code without stopping (Erlang feature)
Realistic Testing: Simulate 1000s of users on one machine
This is production-grade architecture - companies like WhatsApp use this pattern to handle billions of messages!

🔧 Next Steps
Create main.gleam to wire everything together
Add metrics tracking for performance reporting
Implement Zipf properly in user join patterns
Add more realistic timing (delays between actions)
Test at scale (1000+ simulated users)
Want me to create the main entry point or add metrics tracking? 🚀